

HELLENIC MEDITERRANEAN UNIVERCITY

FACULTY OF ENGINEERING

DEPARTMENT OF ELECTRICAL AND COMPUTER ENGINEERING

PATTERN RECOGNITION

Final Project Report

Name: Konstantinidou Eleni

R.N: TH20025

Semester: 7th

Heraklion,

2022-2023

Data Selection & Inspection

Beginning our final project of this course, it is mandatory to select a dataset from the two given ones. We have decided to choose the data regarding the ionosphere, dataset name: “ionosphere.data”.

Firstly, we begin the process by importing all the necessary libraries and then by loading our chosen data (ionosphere dataset) to the notebook. We create a new row with our feature names and we plot the data to check if it’s loaded correctly. By checking the data (the table of the dataset) we just plotted as well as the information about the ionosphere dataset we can see that we do not have any missing values. We have the exact given number of instances (351) and the same number of attributes (34 plus the class attribute = 35 total attributes). Furthermore, another method to check if we any missing values to our data is by using the isna() method (Missing values are denoted with NaN: Not a Number). The isna() method returns a DataFrame object where all the NaN (missing) values are replaced with a Boolean value True and otherwise False. So we have double checked that there are no missing data into our dataset.

Moving on, we note that the class category of the pattern, which contains information whether there is evidence of some type of structure in the ionosphere or not, is embedded inside the data. Therefore, we are going to separate this particular column and create a new different table that holds these values. Also, we created a new table where there are stored all our main data features, excluding the class category. Our last step of the data inspection is to check our data features (columns). With the help of the .names file, that contains all the relevant information about our dataset, we note that is are no unnecessary information about the classifiers so we do not remove any data. It is necessary to note that, the only point where we could have removed information is after checking the first two columns of the dataset. The A01 that contains only ones and the A02 that contains only zeros are two columns that we could have removed from the table but after inspecting the information sheet and seeing that instances in this database are described by two attributes per pulse number, corresponding to the complex values returned by the function resulting from the complex electromagnetic signal, we evaluate them as significant and therefore we let them be as is.

Data preprocessing

The first step of preprocessing our dataset is by boxplots and descriptive statistics. In descriptive statistics, a boxplot diagram is a method for plotting numerical data sets through their quartiles. The end of the box represent the lower and upper quartiles, while the median value is marked by a line inside the box. We can see that our

boxplots have different sizes, from small to bigger ones, as well as their biggest part is mainly depicted on the positive value side. Any circles (points) above or below the box are our data's outliers. By checking the boxplots of our dataset we can note that there are many outliers in the data columns (features). Outliers are unnecessary and therefore we will remove them simply by adding the showfliers argument as False in the function call.

Next, we transform our data features into [0-1] range. First we transform the table with our main data features and then we transform the class category. Regarding the class category we have to remove the letters "g" and "b" and replace them with numbers so that we can , later on, use this information without a problem. In order to achieve that we have set the zero value equal with the letter "g" and the value one equal with the letter "b". Now all the letters in the class category are replaced with zeros and ones (also 0-1 range) representing each category "good" or "bad" accordingly. This step also includes the data normalization of the array.

After we have limited our data range into [0-1] range we can plot them by using a scatter plot. We can see that the data representing the first two columns are more compact while the data of the rest categories are spread and scattered more widely. Therefore, we can consider that some of our best features are the first two columns of the dataset.

Our last step of this preprocessing task is to divide our normalized data into training and test set. The train-test split procedure is used to estimate the performance of machine learning algorithms when they are used to make predictions on data as well as a method to measure the accuracy of our model. After we locate the data into our normalized arrays we split the normalized dataset into 70% of training and 30% of test set. We print the shapes of the training and test data in order to check that everything went fine and we plot individually each training and test set to check the results.

Classification

After we have divided our normalized data into the training and test set we can now implement a linear and a quadratic classifier using the linear and quadratic discriminant analysis respectively. As for the linear discriminant analysis, we begin by fitting the linear discriminant model to the training data. We make predictions on the test data 9 (info which will be used later for assessment of model accuracy) and we generate a classification report where all the evaluation metrics such as the classification error and precision are being included. Regarding the quadratic discriminant analysis, we follow the same steps as with the linear analysis (with the correct libraries of course). By observing the score point in our two analysis reports we can see that the quadratic classifier has higher scoring values that the linear one.

In order though be able to compare the performance of each classifier we will try a few more ways of separating the normalized data in the training and test data division. Our first split is according to these percentages: 80% training and 20% test set , while the second division of our normalized data is according to 90% training and only 10% test set. After these divisions, we do the exact same procedure as before, implementing again both the linear and the quadratic discriminant analysis to this data.

Comparing the results that we get from the classification reports we note that in the 80-20% data split the f1-score value regarding the linear classifier is higher than the same classifier's 70-30% split while as for the 90-10% split we get the same scoring value with the 80-20% split. Regarding the quadratic classifiers, we note that the scoring value is decreasing, with the 90-10% split having the smallest scoring value of the three cases and the 80-20% split coming to the second place. Considering all the above we can say that, because of the quadratic classifier having the smallest values, is considered to be better and offer more accurate results compared to the linear one.

K-means & Hierachical Clustering

Afterwards, we deploy the elbow method in order to estimate the number of clusters. The elbow method is used to determine the optimal number of clusters in K-means clustering and plots the value of the cost function produced by different values of K. After visualizing the data and finding the number of the clusters that we need (clusters = 4 in our case) we deploy the Silhouette analysis. Silhouette analysis can be used to study the separation distance between the resulting clusters. The silhouette plot displays a measure of how close each point in one cluster is to points in the neighboring clusters and thus provides a way to assess parameters like number of clusters visually. This measure has a range of [-1, 1]. After visualizing our data with the silhouette analysis we continue to the deployment of the K-means. We deploy the k-means and compute k-means clustering. We predict the closest cluster each sample belongs to, define the coordinates of cluster centers and finally we scatter plot two random normalized data columns (features) as well as the four cluster centers that we've found in the previous step.

The next step is to deploy the linkage. We begin by deploying the linkage (hierarchical clustering) with the ward method. This is an iterative process which minimizes the total within cluster variance. At each step, the pair of clusters with minimum between cluster distance are merged. We continue by visualizing the linkage results in a dendrogram. It is necessary to stress that each level of the dendogram represent a clustering of the our dataset. At the x-axis of the figure the data points are being presented while at the y-axis the cluster distance.

Naïve Bayes

Naïve Bayes is a probabilistic classifier, which means it predicts on the basis of the probability of an object. It is mainly used in text classification that includes a high-dimensional training dataset. In our project, we continue by implementing first the Naive Bayes model on the normalized data. Afterwards, we continue by implementing the bayes model on each training sets that we have found in the previous steps. We predict the response of each case test dataset and we find the accuracy of the classifier.

Comparing all the results that we have gathered, we note that the predicted accuracy at the bayes classification model is actually higher than the actual results but there is no big deviation between them.

Conclusion

After comparing the performances both of the supervised and the unsupervised learning methods we can say that supervised learning techniques are more accurate and give better results. Although unsupervised learning models can perform more complex tasks than the supervised learning models, they are more unpredictable. Supervised learning model takes direct feedback in order to check if it is predicting the correct output or not while the unsupervised model does not take any feedback at all. Furthermore, the supervised learning model predicts the output and the main goal is to train the model so that it can predict the output when it is given new data. The goals with supervised and unsupervised learning is different. While the supervised is about the prediction of outcomes for new data that is introduced, the other is about getting new insights from massive amounts of new data. In supervised learning, the user will know what results they can expect, whereas in unsupervised learning, they hope to discover something new and unknown.